COURSE-BASED PROJECT DOCUMENTATION | DOC-01

Project Overview & Foundations

System Overview, Context-Free Grammar, & Scope Boundaries

1. Project Title & Designation

Official Title: Java 21 Syntax Checker using GNU Flex, GNU Bison LALR(1), and Python Tkinter.

Academic Track: Course-Based Project (CBP) / Compiler Design & Formal Languages.

2. Executive Abstract

Modern programming languages evolve continuously, adding high-level syntactic sugar and pattern-matching constructs. The Java 21 Syntax Checker is a high-precision, front-end compiler tool designed to formally validate the grammatical correctness of Java 21 (LTS) source files. Built using GNU Flex 2.6.x (lexical DFA scanner) and GNU Bison 3.8.x (LALR(1) shift-reduce parser), the engine compiles to a native Windows DLL (javacheck.dll) and standalone CLI binary (javacheck.exe). An interactive Python 3.10+ / Tkinter desktop interface with an in-memory ctypes foreign function interface (FFI) enables real-time verification, line-number gutter synchronization, Command Palette quick actions, and instant red error-line highlighting. The tool achieved a 100% pass rate across 59 parser tests and 14 GUI tests.

3. What the Project Does

[1] Tokenizes Java 21 source code into lexical tokens with line/column coordinates via Flex.

[2] Parses the token stream against a Java 21 Context-Free Grammar using Bison LALR(1).

[3] Reports whether the source code conforms to the formal Java 21 specification.

[4] Pinpoints the exact source line where the first syntax error occurs.

[5] Presents a responsive, dark-themed VS Code-style graphical developer interface.

4. Why the Project Was Developed (Academic Motivation)

Most educational compilers and literature examples stop at Java 1.4 or Java 7, omitting modern language structures. Java 21 LTS introduced groundbreaking features?such as Record Patterns, Pattern Matching for Switch, Switch Guards (when clauses), Sealed Classes, and Text Blocks. Implementing these modern constructs in a deterministic LALR(1) framework demonstrates advanced grammar disambiguation, precedence engineering, and native systems interoperability.

5. Project Boundary: Syntax Checker vs. Full Compiler

SCOPE & THEORETICAL BOUNDARY

CRITICAL DISTINCTION: This project is strictly a FRONT-END SYNTAX CHECKER and PARSER. It does NOT perform semantic analysis (type checking, symbol resolution, undeclared variable detection), does NOT generate JVM bytecode (.class files), and does NOT execute or debug Java programs.

Compiler Phase	Status in This Project	Implementation Mechanism
1. Lexical Analysis	Fully Implemented (100%)	GNU Flex 2.6.x (DFA, YY_USER_ACTION)
2. Syntax Analysis	Fully Implemented (100%)	GNU Bison 3.8.x (LALR(1) Shift-Reduce)
3. Semantic Analysis	Out of Scope	Not implemented (No symbol tables/types)
4. Intermediate Code Gen	Out of Scope	Not implemented (No TAC or AST)
5. Code Optimization	Out of Scope	Not implemented
6. Target Code Generation	Out of Scope	Not implemented (No .class bytecode)